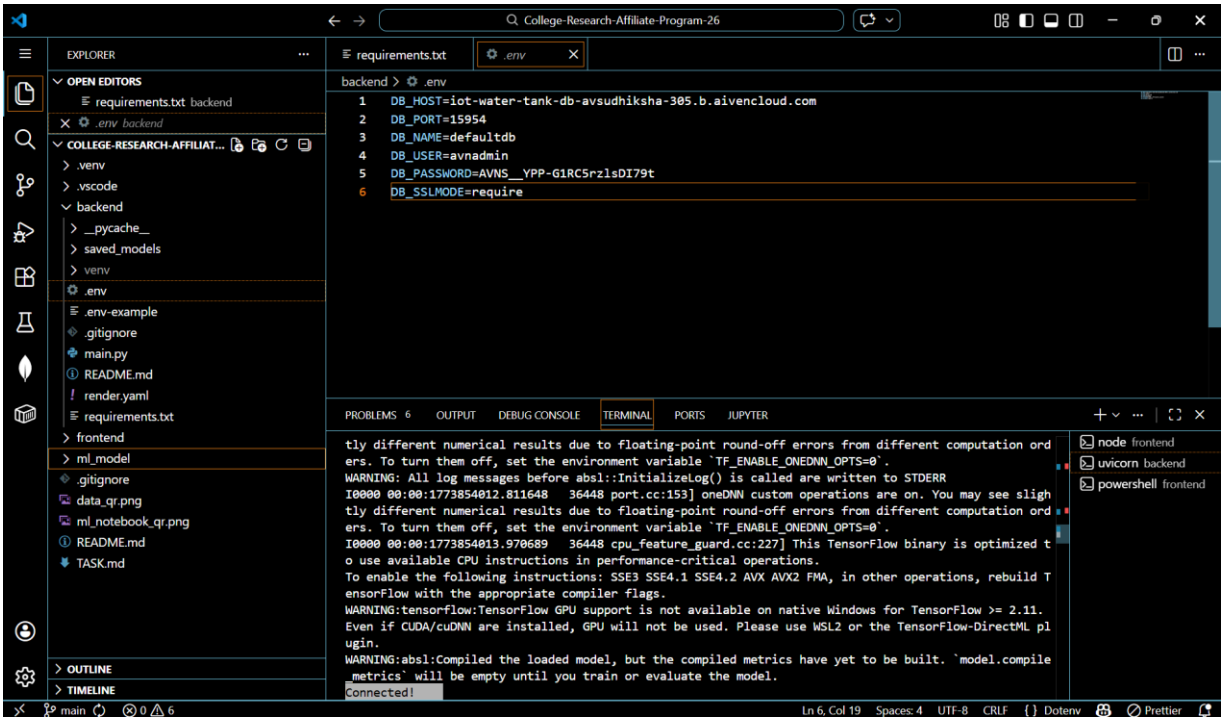
IoT Water Tank Monitoring System with ML Integration

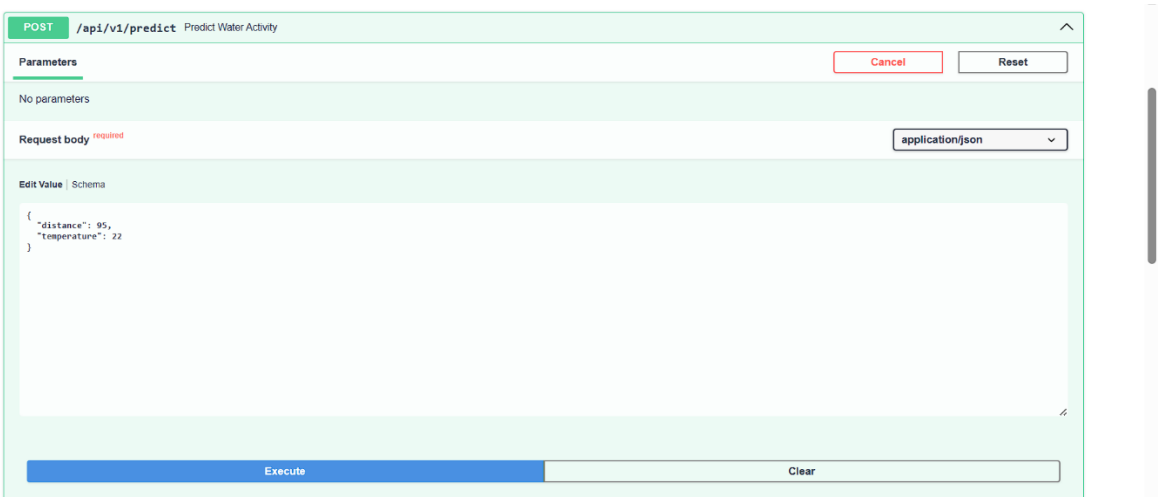
College Research Affiliate Program 2026

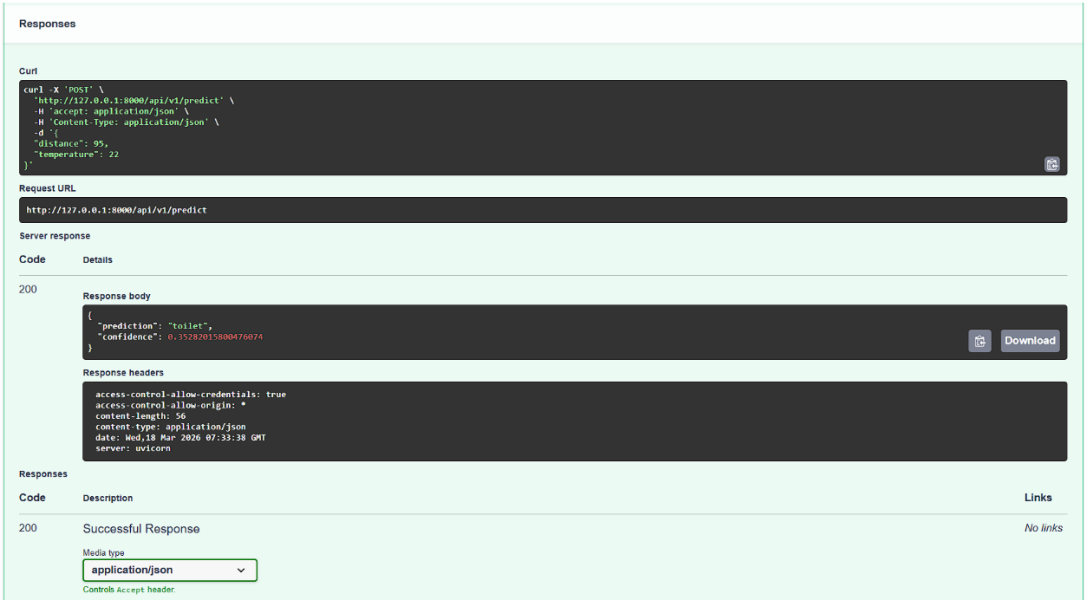
A.V. Sudhiksha 23AG1A7204 avsudhiksha@gmail.com ACE Engineering College

1. Database Connection

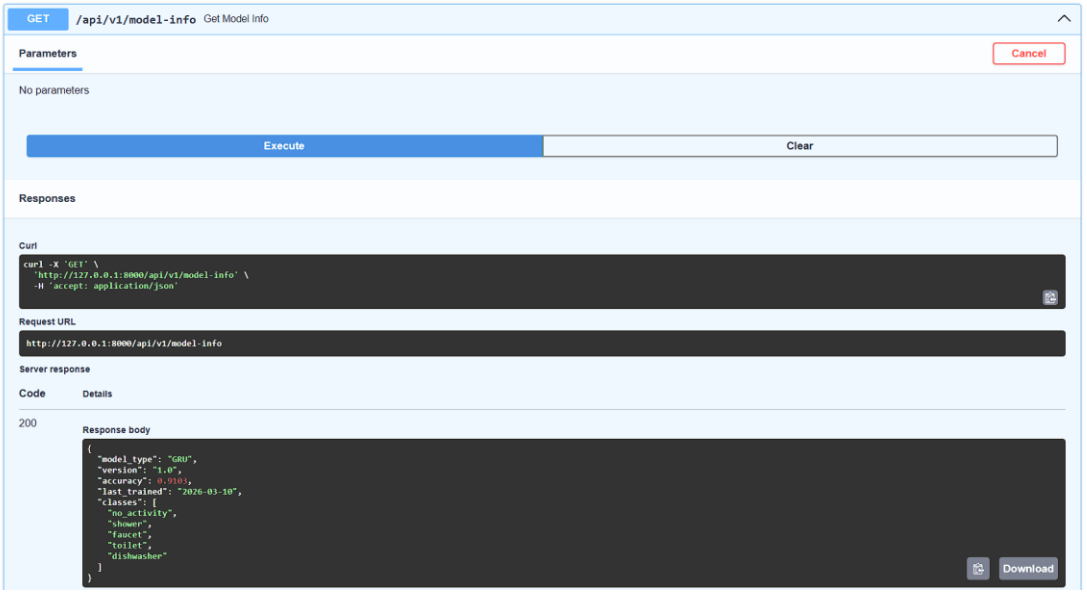


2. Working Prediction API





3. Working Model Info API



4. Code snippet of my prediction implementation

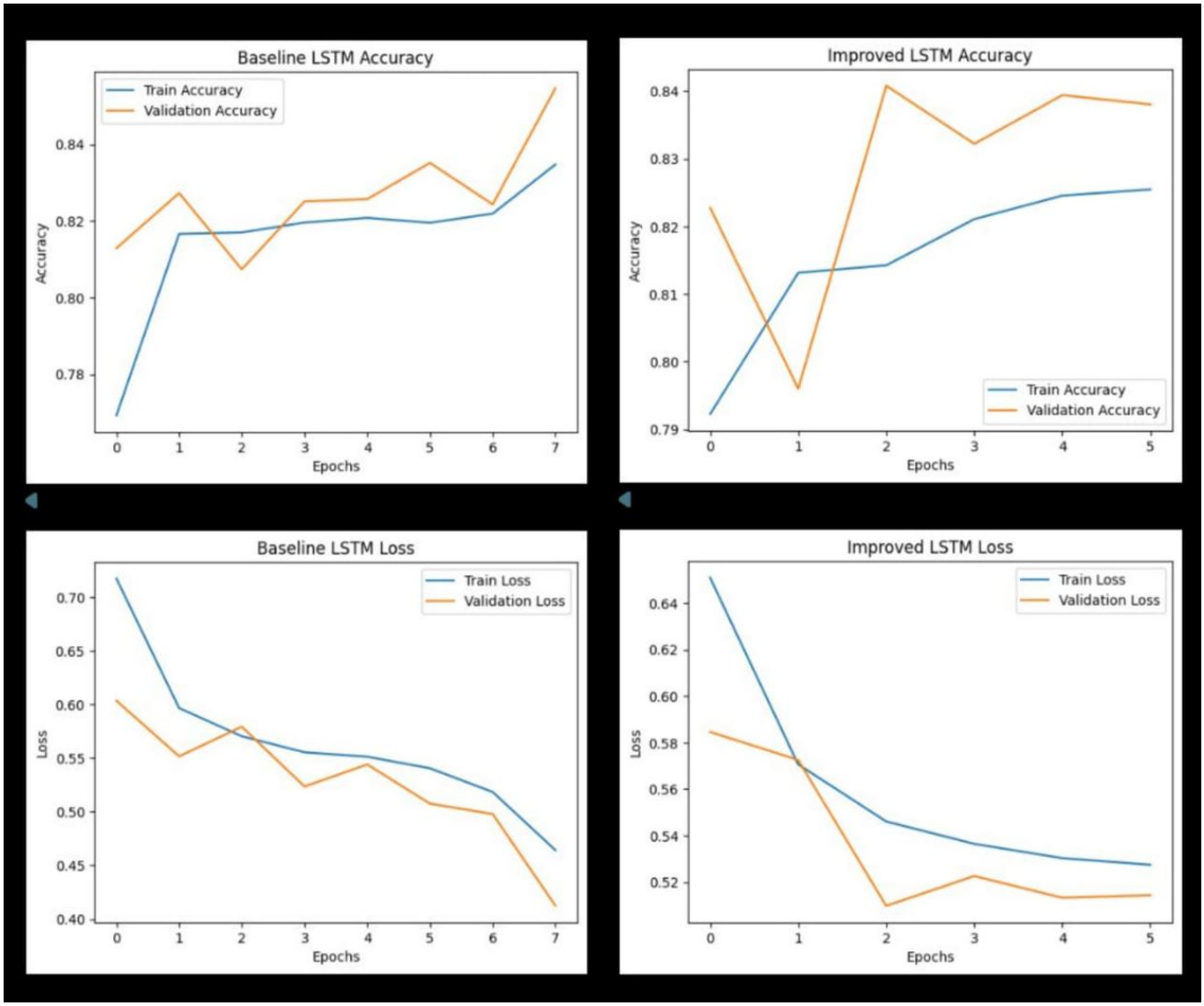
```python
# ==============================
# POST PREDICT API
# ==============================

@app.post("/api/v1/predict")
def predict_water_activity(data: dict):
    try:
        conn = get_connection()
        cur = conn.cursor()

        cur.execute("""
            SELECT field1, field2
            FROM sensor_data
            ORDER BY created_at DESC
            LIMIT 30
        """)
        rows = cur.fetchall()

        if len(rows) < 30:
            cur.close()
            conn.close()
            return {"error": f"Need 30 data points for prediction. Currently have {len(rows)}."}

        sequence = np.array(rows[::-1])

        input_data = sequence.reshape(1, 30, 2).astype('float32')

        prediction = ml_model.predict(input_data)
        predicted_index = np.argmax(prediction)
        predicted_class = classes[predicted_index]
        confidence = float(np.max(prediction))

        cur.execute("""
            INSERT INTO predictions (distance, temperature, prediction, confidence)
            VALUES (%s, %s, %s, %s)
        """, (float(sequence[-1][0]), float(sequence[-1][1]), predicted_class, confidence))

        conn.commit()
        cur.close()
        conn.close()

        return {
            "prediction": predicted_class,
            "confidence": confidence,
            "status": "success"
        }

    except Exception as e:
        return {"error": str(e)}
```

5. Training Log Table

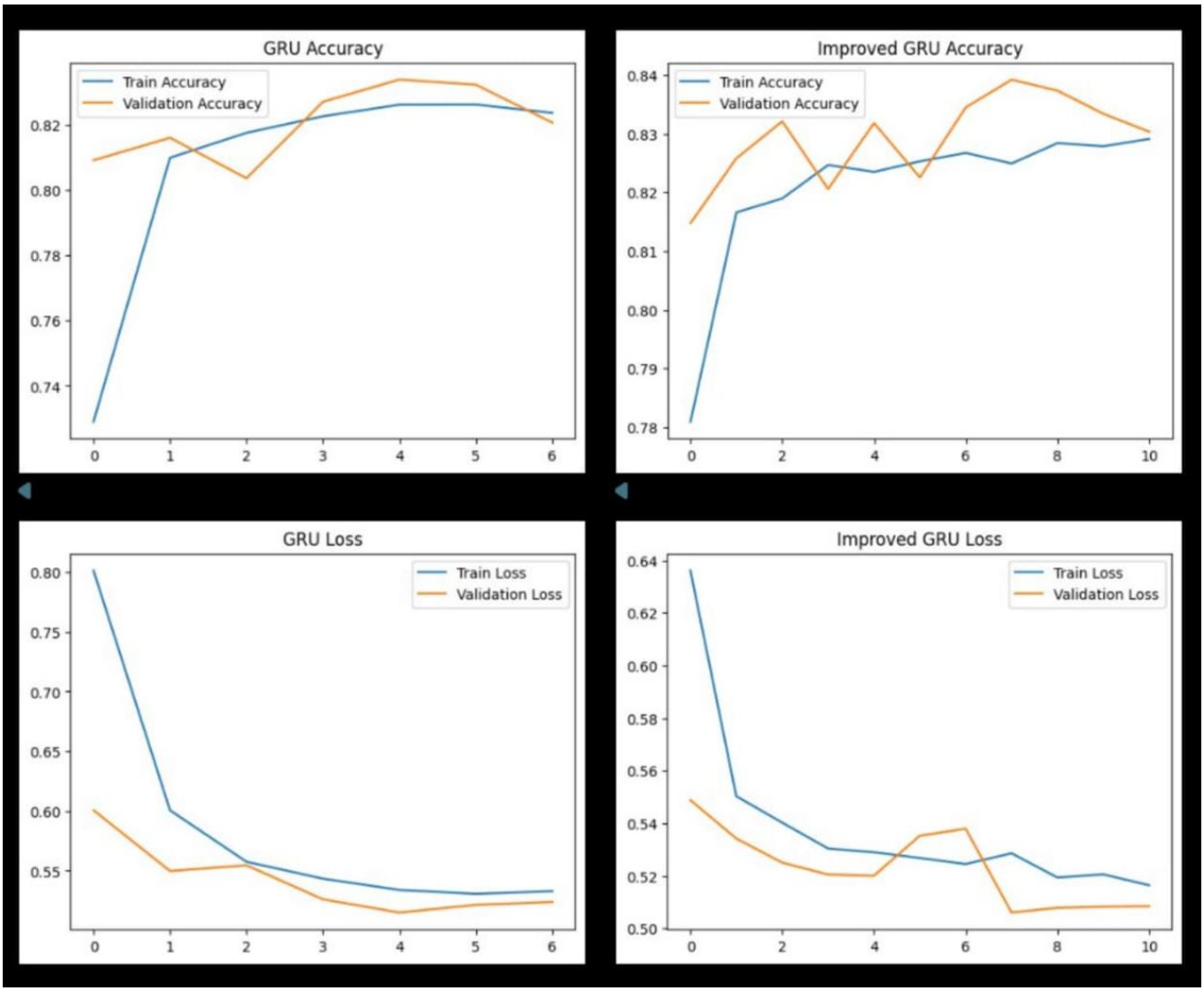
```
Training Experiments

| Experiment | Model            | Layers | Units        | Dropout | Learning Rate | Epochs | Accuracy (%) | F1 Score | Notes                    |
|------------|------------------|--------|--------------|---------|---------------|--------|--------------|----------|--------------------------|
| 1          | LSTM             | 2      | 48,24        | 0.2     | 0.001         | 8      | 83.3         | 0.64     | Baseline                 |
| 2          | Improved LSTM    | 3      | 128,64,32    | 0.3     | 0.001         | 20     | 83.8         | 0.64     | Slight improvement       |
| 3          | GRU              | 2      | 48,24        | 0.2     | 0.001         | 8      | 84.8         | 0.61     | Baseline                 |
| 4          | Improved GRU     | 3      | 128,64,32    | 0.3     | 0.001         | 20     | 91.0         | 0.84     | Best model               |
| 5          | CNN              | 2      | 48,24        | 0.2     | 0.001         | 8      | 85.2         | 0.75     | Baseline                 |
| 6          | Improved CNN     | 2      | 64,128       | 0.3     | 0.001         | 20     | 86.9         | 0.64     | Accuracy ↑, F1 ↓         |


During initial experiments, GRU showed the best performance and was selected for deployment.
After re-running experiments, CNN showed slightly better accuracy due to randomness in training.
However, GRU was retained as it is more suitable for sequential time-series modeling.
```

6. Training accuracy/loss curves

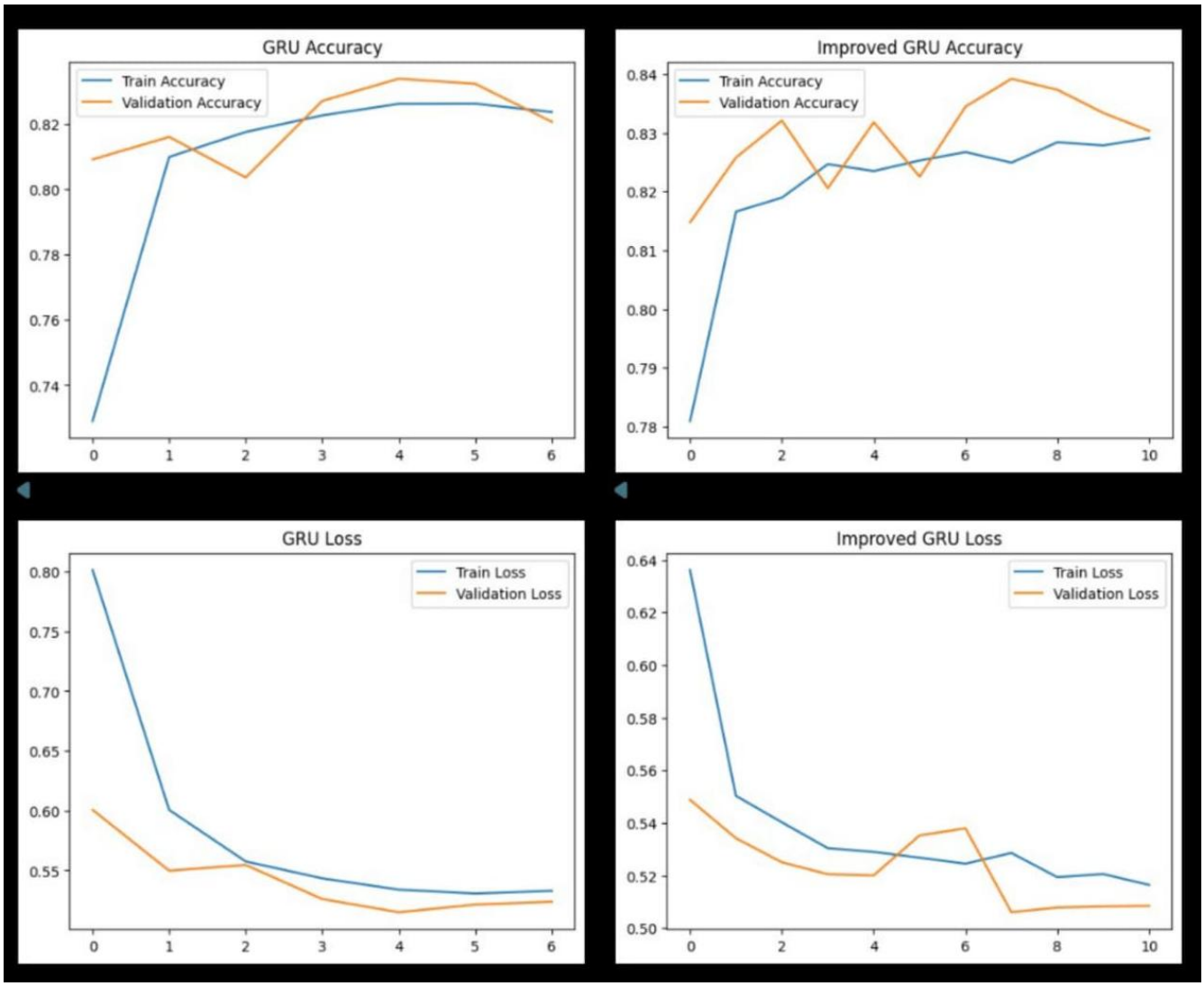
6.1. LSTM Model



6.2. GRU Model


GRU Accuracy
Train Accuracy
Validation Accuracy
Improved GRU Accuracy
Train Accuracy
Validation Accuracy
GRU Loss
Train Loss
Validation Loss
Improved GRU Loss
Train Loss
Validation Loss

6.3. CNN Model



7. Final model accuracy

Used GRU Model:

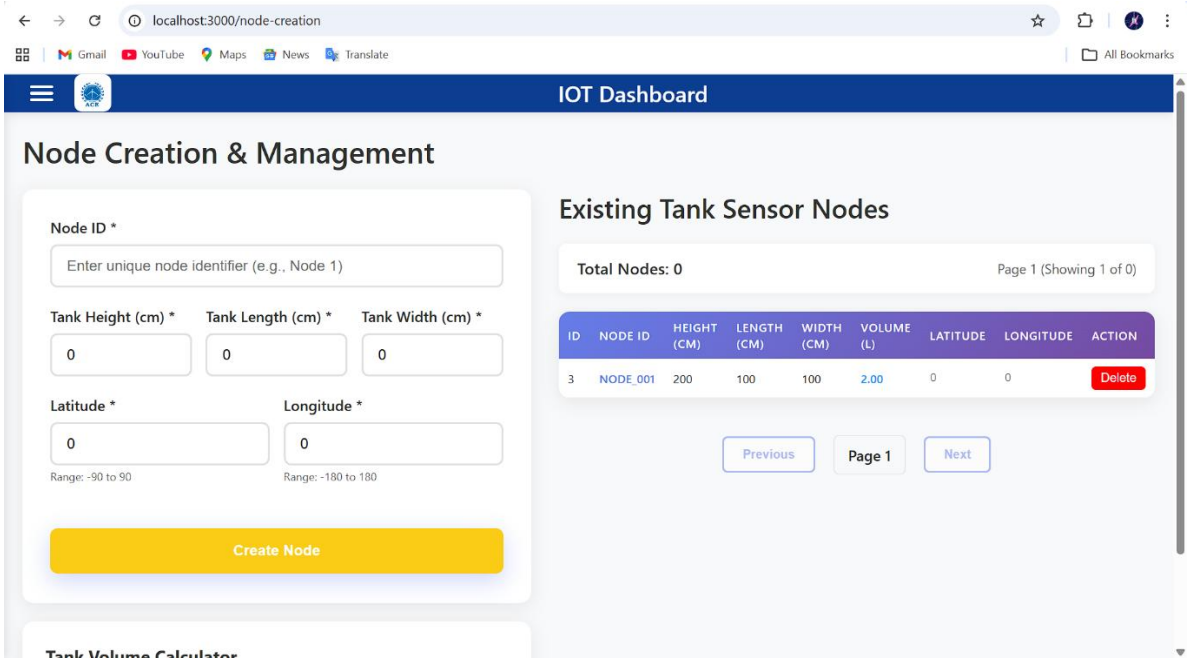
Baseline Model Accuracy: 84.8%

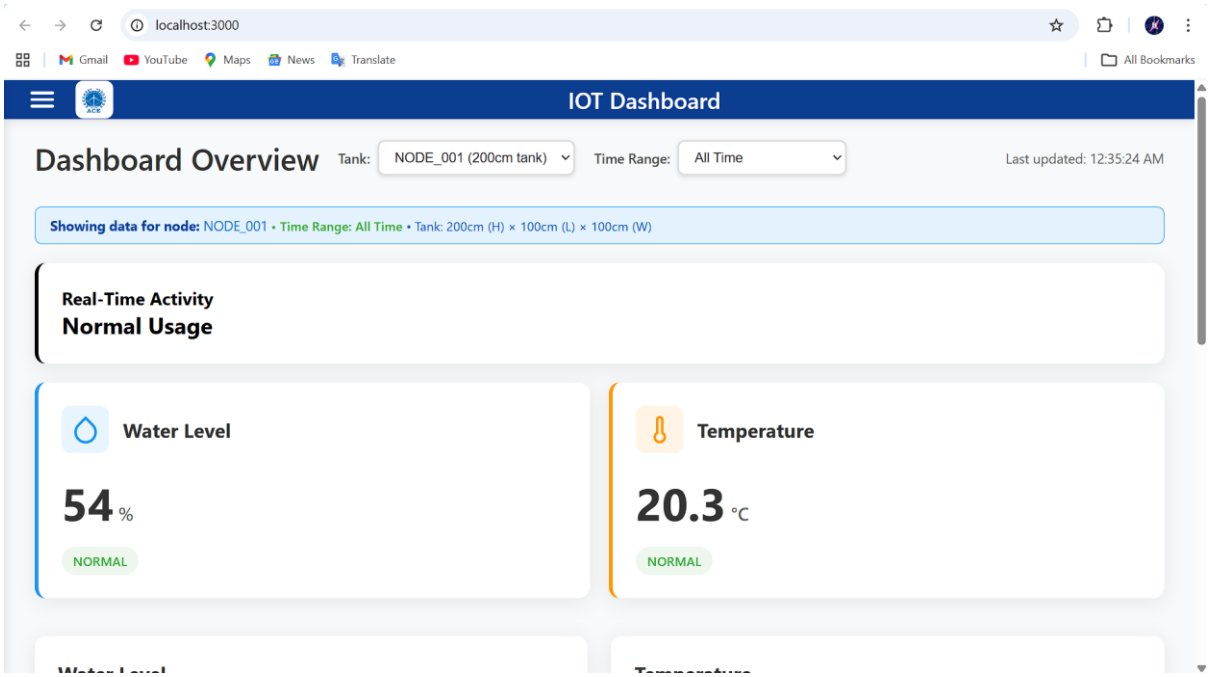
Improved Model Accuracy: 91.03%

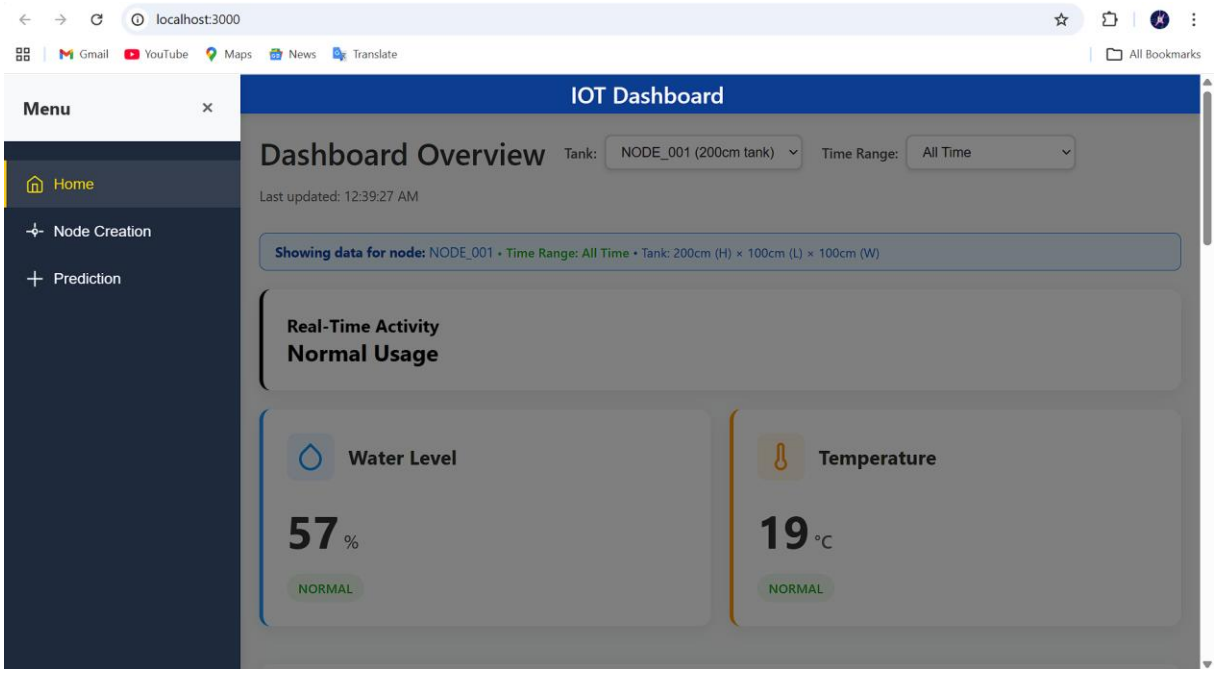
Accuracy Improvement: +7%

The deep learning model (GRU) significantly outperformed traditional ML models by capturing temporal patterns in water usage data.

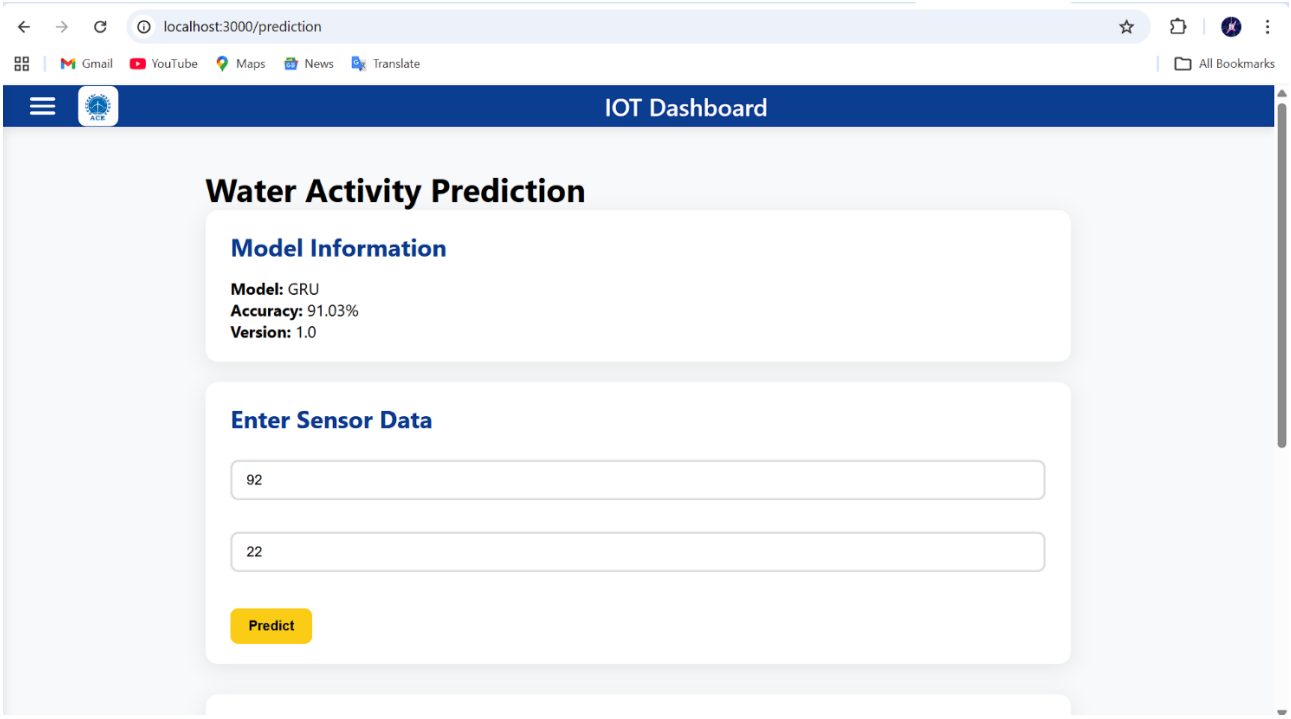
8. Dashboard with College Logo and Colors

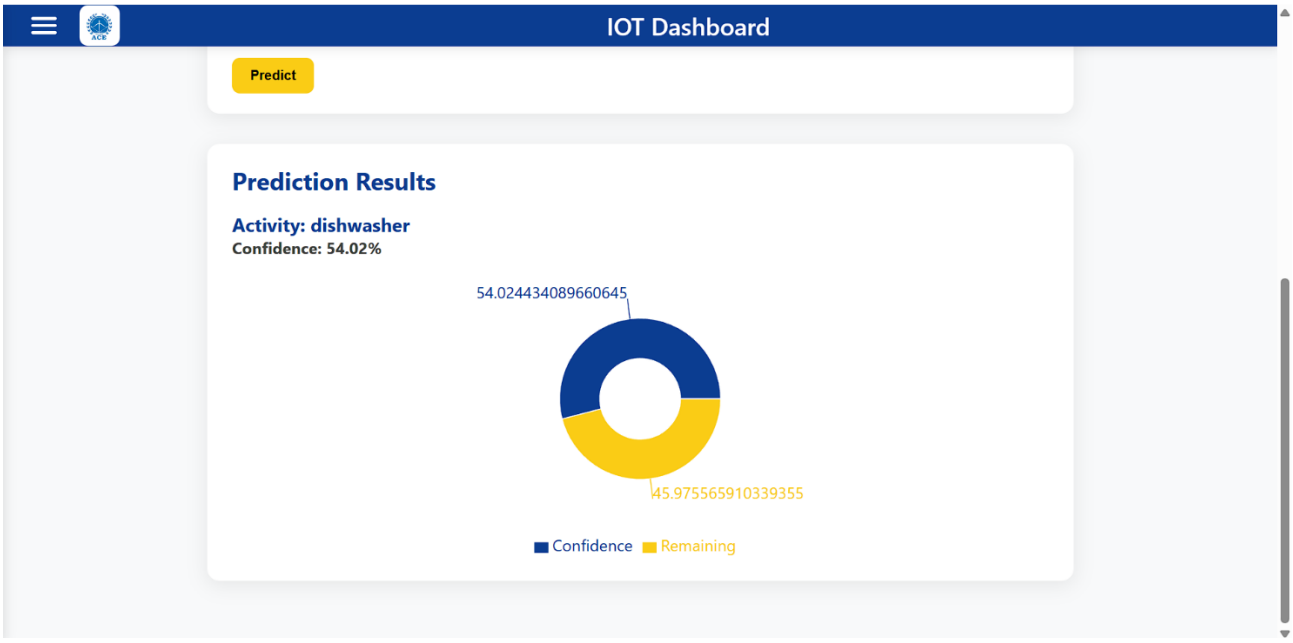




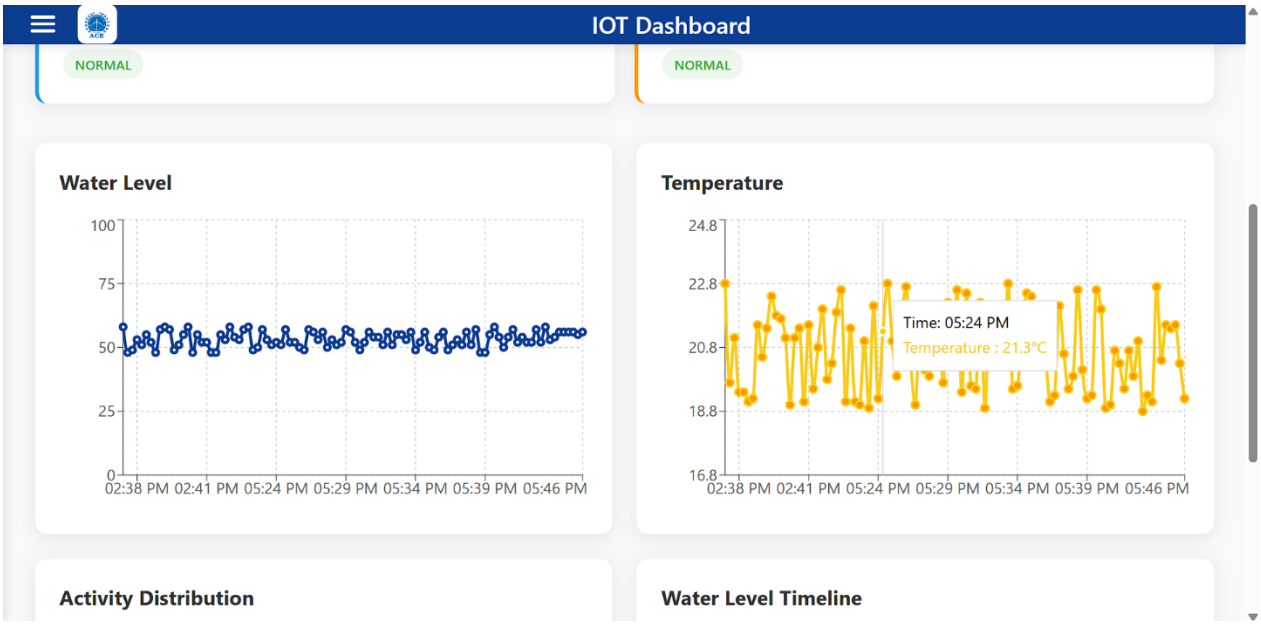

localhost:3000
Gmail
YouTube
Maps
News
Translate
All Bookmarks
Menu
IOT Dashboard
Home
Node Creation
Prediction
Dashboard Overview
Tank:
NODE_001 (200cm tank)
Time Range:
All Time
Last updated: 12:39:27 AM
Showing data for node: NODE_001 • Time Range: All Time • Tank: 200cm (H) × 100cm (L) × 100cm (W)
Real-Time Activity
Normal Usage
Water Level
57 %
NORMAL
Temperature
19 °C
NORMAL

9. Prediction page with working predictions





10. New custom charts



11. Code snippets of key modifications

Navbar.js

frontend > src > components > Navbar.js > default

```
import React from 'react';
import collegeLogo from '../assets/college-logo.png';

const Navbar = ({ onToggleSidebar }) => {
  return (
    <nav className="navbar">
      <div className="navbar-left">
        <button className="hamburger-btn" onClick={onToggleSidebar}>
          <div className="hamburger-line"></div>
          <div className="hamburger-line"></div>
          <div className="hamburger-line"></div>
        </button>

        <div className="logo">
          <img src={collegeLogo} alt="College Logo" className="logo-img" />
        </div>
      </div>

      <div className="navbar-center">
        <h1 className="navbar-title">IOT Dashboard</h1>
      </div>
```

App.css

frontend > src > App.css > ...

```
:root {
  --primary-color: #0b3d91;
  --secondary-color: #1e293b;
  --accent-color: #facc15;
}
```

Prediction.js

frontend > src > pages > Prediction.js > Prediction

```
const Prediction = () => {
  // Call prediction API
  const handlePredict = async () => {
    try {
      const res = await axios.post(`${config.API_BASE_URL}/api/v1/predict`, {
        distance: parseFloat(inputData.distance),
        temperature: parseFloat(inputData.temperature)
      });
```

```
JS Home.js 6 ×
frontend > src > pages > JS Home.js > ...
16  const Home = () => {
576
577             <div className="graph-card">
578               <h3>Activity Distribution</h3>
579               <ResponsiveContainer width="100%" height={300}>
580                 <PieChart>
581                   <Pie
582                     data={getDistributionData()}
583                     dataKey="value"
584                     outerRadius={100}
585                   >
586                     {getDistributionData().map((entry, index) => (
587                       <Cell key={index} fill={CHART_COLORS[index % CHART_COLORS.length]} />
588                     ))}
589                   </Pie>
590                   <Tooltip />
591                   <Legend />
592                 </PieChart>
593               </ResponsiveContainer>
594             </div>
595
596             <div className="graph-card">
597               <h3>Water Level Timeline</h3>
598               <ResponsiveContainer width="100%" height={300}>
599                 <BarChart data={waterLevelData}>
600                   <CartesianGrid strokeDasharray="3 3" />
601                   <XAxis dataKey="time" />
602                   <YAxis />
603                   <Tooltip />
604                   <Bar dataKey="value" fill="var(--primary-color)" />
605                 </BarChart>
606               </ResponsiveContainer>
607             </div>
608           </div>
609         </div>
610       );
611     };
```

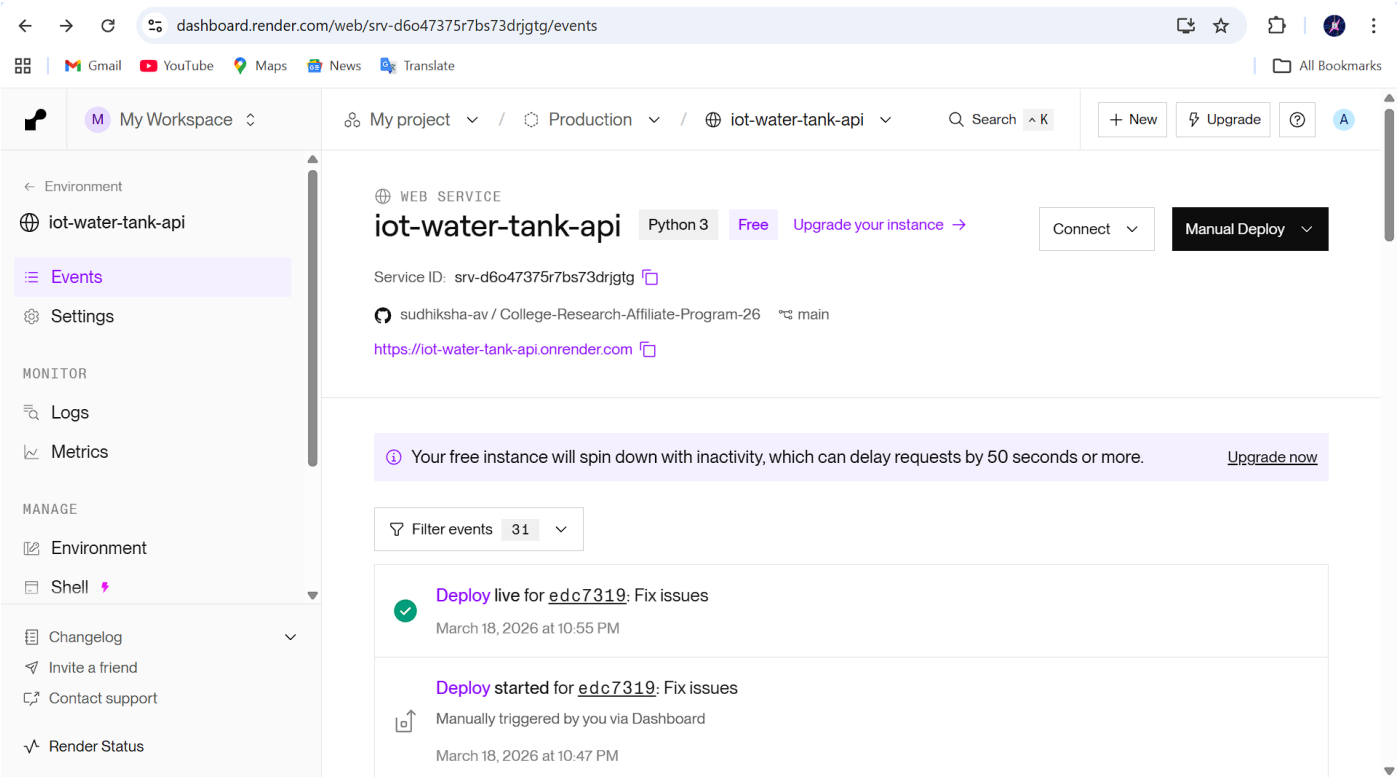
12. URLs

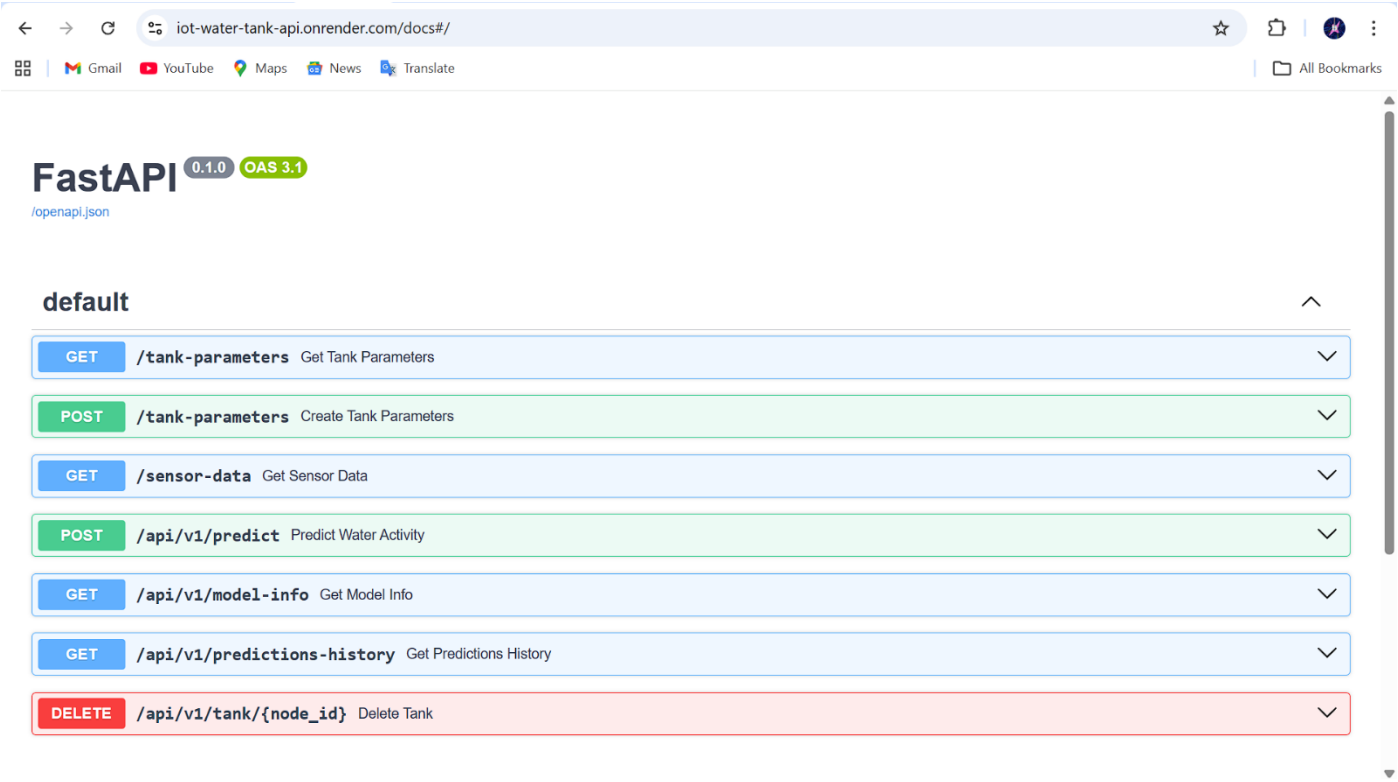
12.1. Deployed backend URL: https://iot-water-tank-api.onrender.com

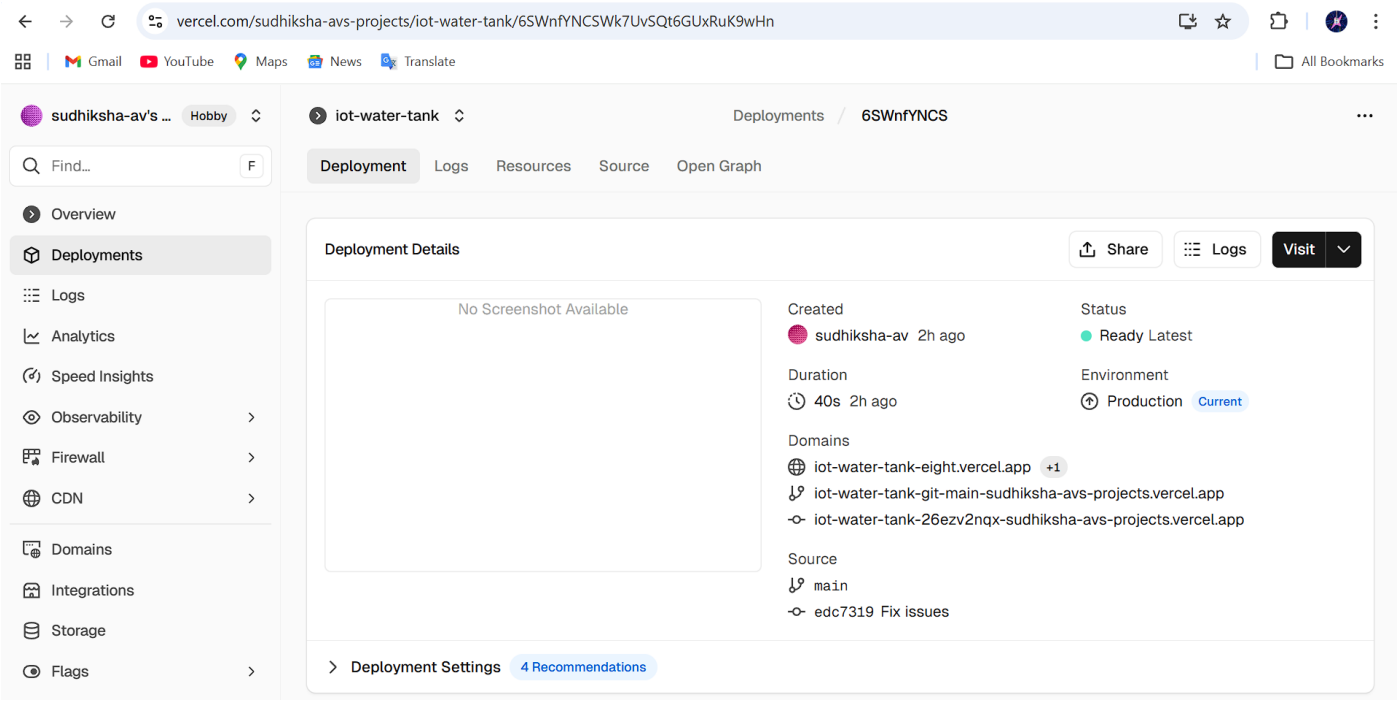
12.2. Deployed frontend URL: https://iot-water-tank-sudhiksha-avs-projects.vercel.app/

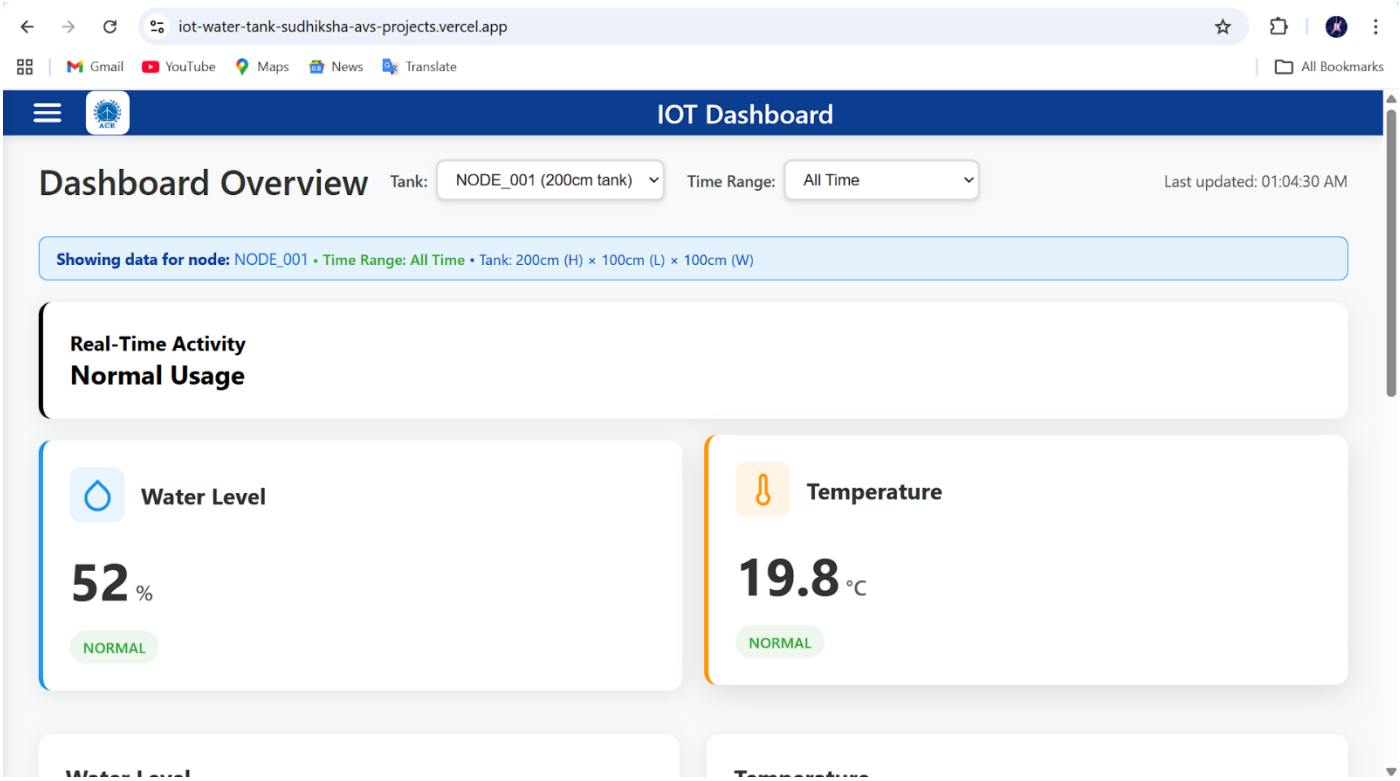
12.3. GitHub URL: https://github.com/sudhiksha-av/College-Research-Affiliate-Program-26.git

13. Working deployed application

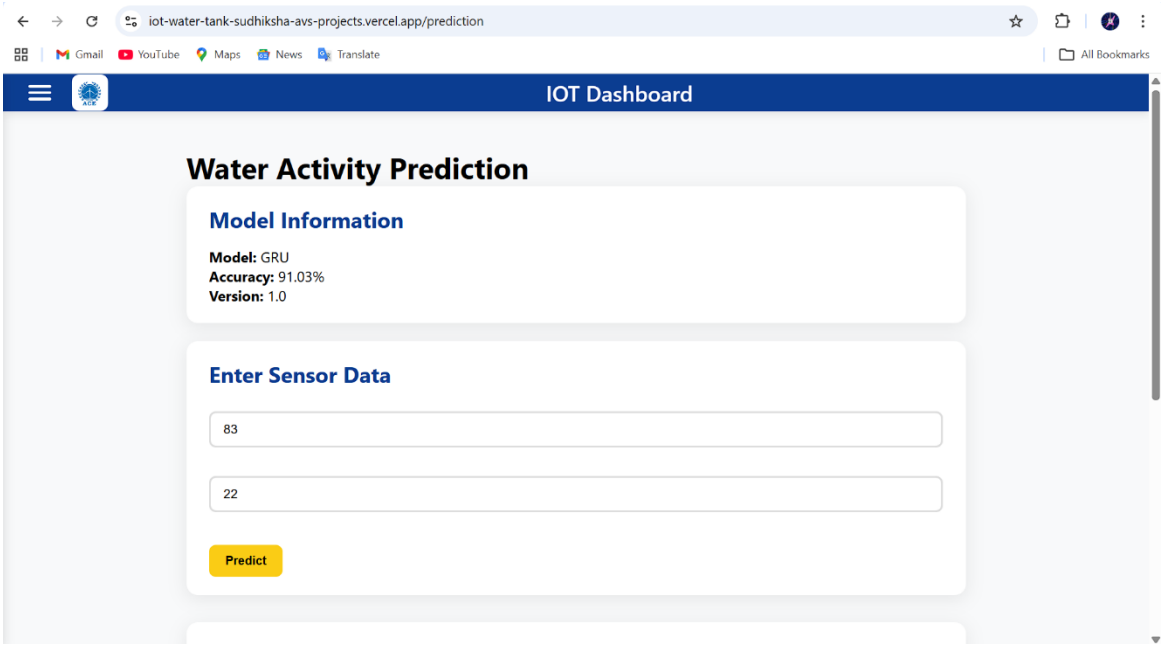


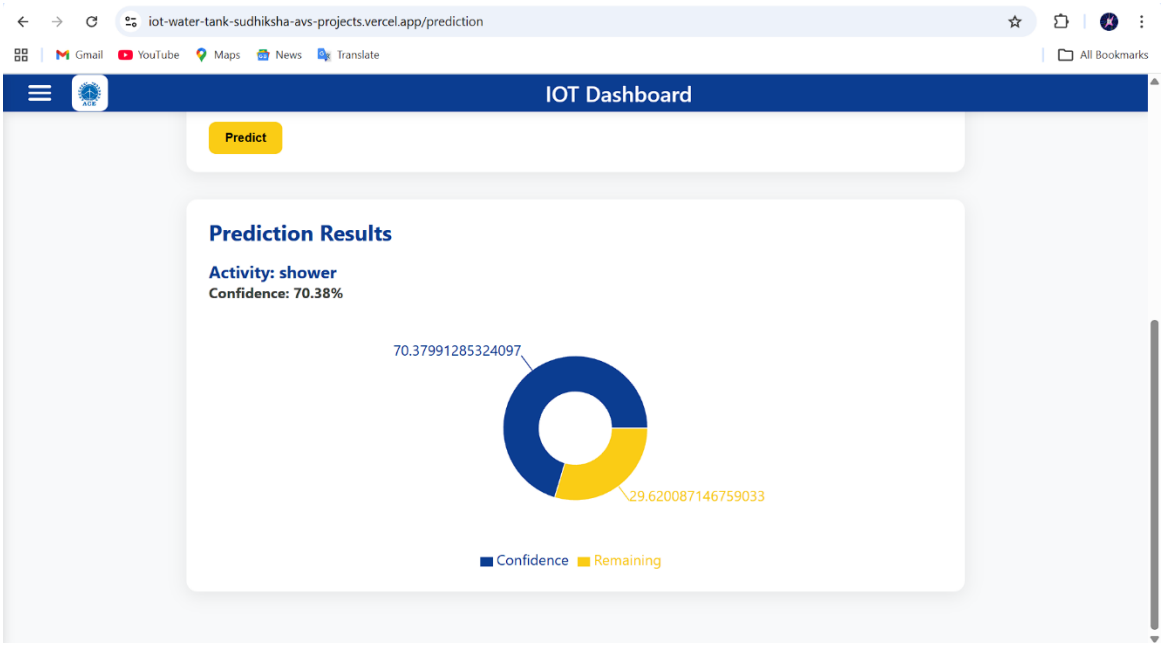



vercel.com/sudhiksha-avs-projects/iot-water-tank/6SWnfYNCSWk7UvSQt6GUxRuK9wHn
Gmail
YouTube
Maps
News
Translate
All Bookmarks
sudhiksha-av's ...
Hobby
iot-water-tank
Deployments
6SWnfYNCS
Find...
F
Deployment
Logs
Resources
Source
Open Graph
Overview
Deployments
Logs
Analytics
Speed Insights
Observability
Firewall
CDN
Domains
Integrations
Storage
Flags
Deployment Details
Share
Logs
Visit
No Screenshot Available
Created
sudhiksha-av 2h ago
Status
Ready Latest
Duration
40s 2h ago
Environment
Production
Current
Domains
iot-water-tank-eight.vercel.app
+1
iot-water-tank-git-main-sudhiksha-avs-projects.vercel.app
iot-water-tank-26ezv2nqx-sudhiksha-avs-projects.vercel.app
Source
main
edc7319 Fix issues
Deployment Settings
4 Recommendations


iot-water-tank-sudhiksha-avs-projects.vercel.app
Gmail
YouTube
Maps
News
Translate
All Bookmarks
IOT Dashboard
Dashboard Overview
Tank:
NODE_001 (200cm tank)
Time Range:
All Time
Last updated: 01:04:30 AM
Showing data for node: NODE_001 • Time Range: All Time • Tank: 200cm (H) × 100cm (L) × 100cm (W)
Real-Time Activity
Normal Usage
Water Level
52 %
NORMAL
Temperature
19.8 °C
NORMAL

14. Prediction working on deployed app





15. ML Improvements Made and Accuracy Achieved

In this project, multiple deep learning models were developed and improved to predict water usage activities based on sensor data (distance and temperature).

Baseline Models Implemented:

- LSTM
- GRU
- CNN

Improvements Applied:

- Increased number of units in layers
- Added multiple hidden layers
- Introduced Dropout layers for regularization
- Tuned learning rate and optimizer
- Improved architecture depth (especially GRU)

Final Results:

Model	Accuracy (%)	F1 Score
LSTM	83.33%	0.640
Improved LSTM	83.80%	0.644
GRU	84.87%	0.619
Improved GRU	**91.03%**	**0.845**
CNN	85.29%	0.759
Improved CNN	86.98%	0.644

Final Selection:

The Improved GRU model was selected as the best-performing model with:

- Highest accuracy: 91.03%
- Best generalization (F1 score)

16. API Endpoints added

API	Method	Description
/predict	POST	Predict water activity
/model-info	GET	Get model details
/predictions-history	GET	View past predictions
/tank/{node_id}	DELETE	Delete a tank

17. Frontend Features Added

The frontend was built using React.js with a modern dashboard UI.

UI Enhancements:

- Added college branding (ACE Engineering College)
- Custom color theme using CSS variables
- Integrated college logo in Navbar

Dashboard Features:

- Real-time water level and temperature display
- Interactive line charts using Recharts
- Node (tank) selection dropdown
- Added delete option in nodes table
- Time range filtering (1h, 6h, 24h, custom)

Prediction Page Features:

- Input form for sensor values
- API integration for predictions
- Display of predicted activity
- Confidence Pie Chart visualization
- Centered and styled chart for better UI

Additional Enhancements:

- Added animations to charts
- Improved responsiveness
- Better data handling for no-data cases

18. Challenges Faced and Solutions

1. TensorFlow Deployment Issues

Problem:

- TensorFlow was not compatible with latest Python versions on Render
- Model loading errors (quantization_config, batch_shape)

Solution:

- Used version TensorFlow 2.21
- Retrained model using compatible version
- Saved model in .h5 format

2. Dependency Conflicts

Problem:

- Conflict between TensorFlow and FastAPI dependencies (typing-extensions)

Solution:

- Carefully selected compatible versions
- Cleaned requirements.txt

3. Model Serialization Errors

Problem:

- Model trained in Keras 3 not loading in TensorFlow 2.x

Solution:

- Re-trained model using TensorFlow Keras (v2.21)
- Ensured same environment for training and deployment

4. **Frontend Integration Errors**

Problem:

- Missing components (Prediction page, logo)
- Routing issues

Solution:

- Created missing files
- Fixed import paths
- Added navigation links

5. **No Sensor Data Issue**

Problem:

- Selected nodes showing no data

Solution:

- Corrected node-to-tank mapping
- Added fallback handling and messages

6. **React Setup Errors**

Problem:

- react-scripts not recognized

Solution:

- Installed dependencies using npm install

7. **Vercel Build Failures**

Problem:

- Frontend deployment failed due to missing environment variables and API URLs

Solution:

- Configured environment variables in Vercel
- Updated API base URLs in frontend config